

UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES



SISTEMAS DE COMPUTACIÓN

Trabajo práctico #1:
“Rendimiento”

Comisión: Única

Docente: Ing. Javier Alejandro Jorge , Ing. Miguel Ángel Solinas

Alumno: Castilla Juan Felipe - Dalmazzo Agustin

Año lectivo: 2026

Cuatrimestre: Segundo

Índice

Benchmarks.....	3
Introducción.....	3
Listado de Benchmarks y Usos.....	3
Rendimiento de Procesadores.....	7
Actividad.....	7
Rendimiento ESP32.....	9
Tabla rendimiento ESP32 Enteros.....	9
El aumento de la frecuencia provoca de manera lineal una disminución en el tiempo de ejecución del programa, como se puede observar, duplicar la frecuencia tiene como consecuencia dividir a la mitad el tiempo de ejecución.....	9
Tabla rendimiento ESP32 Flotantes.....	9
El ESP32 incluye una Unidad de Punto Flotante (FPU) por hardware de precisión simple (32 bits). Esto significa que las instrucciones matemáticas con float no requieren ser emuladas por software; el hardware dedicado las resuelve en muy pocos ciclos de reloj, logrando velocidades comparables a las de los enteros. Se observa que al duplicar la frecuencia, se divide a la mitad el tiempo, es un escalamiento lineal.....	10
Tabla rendimiento ESP32 Double.....	10
Time Profiling.....	14
Gráfico.....	18
Perf.....	18
Conclusiones.....	19
IP, Ancho de Banda y Latencia.....	19
IP's de los Equipos.....	19
Medición de Ancho de Banda.....	20
Fast.....	21
Medición de Latencia.....	21

Benchmarks

Introducción

Un **benchmark** es una prueba estandarizada o un conjunto de métricas que se utiliza para evaluar el rendimiento, la calidad o la eficiencia de un sistema, producto o proceso, comparándolo con un estándar o con alternativas similares.

Existen 4 tipos de Benchmarks:

Tipo	Definición
<i>Sintético</i>	Programas creados artificialmente con el propósito exclusivo de medir el rendimiento. Simulan estadísticamente el comportamiento de una carga de trabajo o "estresan" un componente específico del hardware
<i>Reducido</i>	Programas extremadamente pequeños y simples, generalmente de unas pocas docenas o cientos de líneas de código, que ejecutan un algoritmo clásico o una tarea básica. Útiles en las etapas iniciales de diseño de un procesador o para tareas académicas específicas
<i>Kernel/Núcleo</i>	Se trata de extraer únicamente el fragmento de código más pesado y computacionalmente intensivo de una aplicación real (el bucle o ciclo donde el programa pasa el 90% de su tiempo de ejecución) y evaluarlo de forma aislada, ignorando el resto del programa.
<i>Programa real</i>	Consiste en utilizar software comercial o de código abierto completo y exacto al que usan los usuarios todos los días, ejecutando una carga de trabajo predefinida

Listado de Benchmarks y Usos

Listado

- Compilación del Kernel de linux
- Bucles de cálculo
- Pruebas de escalamiento
- Phoronix Test Suite

- Sysbench
- 3DMark
- Stress-ng
- Speedtest
- Fast
- Programa real
- CoreMark

Usos

Tareas	Benchmark
Simulación de circuitos	- CoreMark/Phoronix/Bucles de Cálculo
De calculo y Analisis numerico	- CoreMark/Phoronix/Bucles de Cálculo
Diseño de Placas y 3D	- CoreMark/Phoronix/Bucles de Cálculo
Programación Embebida	- Stress-ng
Videojuegos	- 3DMark - Speedtest
Ofimática	- Programa Real
Desarrollo de Aplicaciones y BD	- Sysbench

Ejecución de CoreMark y Comparativa

```
> fastfetch
```



```

peligro@elpeligroso
-----
OS: Manjaro Linux x86_64
Host: 82YU (Lenovo V15 G4 AMN)
Kernel: Linux 6.18.44-1-MANJARO
Uptime: 3 hours, 14 mins
Packages: 1422 (pacman)[stable]
Shell: zsh 5.9.2
Display (LEN9051): 1920x1080 in 16", 60 Hz [Built-in]
DE: KDE Plasma 6.7.4
WM: KWin (Wayland)
WM Theme: Breeze
Theme: Breeze (BreathLight) [Qt], Breeze [GTK2/3]
Icons: breeze-dark [Qt], breeze-dark [GTK2/3/4]
Font: Noto Sans (10pt) [Qt], Noto Sans (10pt) [GTK2/3/4]
Cursor: breeze (24px)
Terminal: konsole 26.4.3
Terminal Font: Hack Nerd Font Mono (11pt)
CPU: AMD Ryzen 5 7520U (8) @ 4.39 GHz
GPU: AMD Radeon 610M [Integrated]
Memory: 5.40 GiB / 7.08 GiB (76%)
Swap: Disabled
Disk (/): 17.18 GiB / 232.59 GiB (7%) - btrfs
Local IP (wlp2s0): 192.168.1.38/24
Battery (L22M2PG2): 62% [Discharging]
Locale: es_AR.UTF-8

```



```
> cat run1.log
2K performance run parameters for coremark.
CoreMark Size : 666
Total ticks : 12832
Total time (secs): 12.832000
Iterations/Sec : 31172.069825
Iterations : 400000
Compiler version : GCC16.1.1 20260728
Compiler flags : -O2 -DPERFORMANCE_RUN=1 -lrt
Memory location : Please put data memory location here
                  (e.g. code in flash, data on heap etc)
seedcrc : 0xe9f5
[0]crclist : 0xe714
[0]crcmatrix : 0x1fd7
[0]crcstate : 0x8e3a
[0]crcfinal : 0x65c5
Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 31172.069825 / GCC16.1.1 20260728 -O2 -DPERFORMANCE_RUN=1 -lrt / Heap
> cat run2.log
2K validation run parameters for coremark.
CoreMark Size : 666
Total ticks : 12746
Total time (secs): 12.746000
Iterations/Sec : 31382.394477
Iterations : 400000
Compiler version : GCC16.1.1 20260728
Compiler flags : -O2 -DPERFORMANCE_RUN=1 -lrt
Memory location : Please put data memory location here
                  (e.g. code in flash, data on heap etc)
seedcrc : 0x18f2
[0]crclist : 0xe3c1
[0]crcmatrix : 0x0747
[0]crcstate : 0x8d84
[0]crcfinal : 0x8091
Correct operation validated. See README.md for run and reporting rules.
```

```

agustin@HP-Mersa:~/Escritorio/Facultad/Computacion/CoreMark/coremark$ neofetch
      ....-:-:-:-
      .-MMMMMMMMMMMMMMMM-
      .-MMMM`-:-:-:-:-`MMMM-
      .:MMMM.:MMMMMMMMMMMMMMMM.:MMMM.:
      -MMM-M--MMMMMMMMMMMMMMMMMMMM.MMM-
      `:MMM:MM` :MMMM:.....-:-MMMM:MMM:`
      :MMM:MMM` :MM: `:-:-:-:-` :MMM:MMM:
      .MMM.MMMM` :MM. -MM. .MM- `MMMM.MMM.
      :MMM:MMM` :MM. -MM- .MM: `MMMM-MMM:
      :MMM:MMM` :MM. -MM- .MM: `MMMM:MMM:
      :MMM:MMM` :MM. -MM- .MM: `MMMM-MMM:
      .MMM.MMMM` :MM:--MM:--MM: `MMMM.MMM.
      :MMM:MMM- `MMMMMMMMMMMMMM-` -MMM-MMM:
      :MMM:MMM:` :MMM:MMM:
      .MMM.MMMM:-----:MMMM.MMM.
      '-MMMM.-MMMMMMMMMMMMMMMM-.MMMM-'
      '-MMMM`-:-:-:-`MMMM-.'
      '-MMMMMMMMMMMMMM-'
      ``-:-:-:-``

```

```

agustin@HP-Mersa
-----
OS: Linux Mint 22.3 x86_64
Host: HP Laptop 15-ef2126wm
Kernel: 7.0.0-28-generic
Uptime: 4 hours, 9 mins
Packages: 2522 (dpkg), 9 (flatpak)
Shell: bash 5.2.21
Resolution: 1920x1080
DE: Cinnamon 6.6.9
WM: Mutter (Muffin)
WM Theme: Orchis-Dark (Mint-Y)
Theme: Orchis-Dark [GTK2/3]
Icons: Papirus-Dark [GTK2/3]
Terminal: gnome-terminal
CPU: AMD Ryzen 5 5500U with Radeon Graphics (12) @ 4.056GHz
GPU: AMD ATI 03:00.0 Lucienne
Memory: 4548MiB / 7252MiB

```



```

agustin@HP-Mersa:~/Escritorio/Facultad/Computacion/CoreMark/coremark$ cat run1.log
2K performance run parameters for coremark.
CoreMark Size      : 666
Total ticks        : 13649
Total time (secs): 13.649000
Iterations/Sec     : 29306.176277
Iterations         : 400000
Compiler version   : GCC13.3.0
Compiler flags     : -O2 -DPERFORMANCE_RUN=1 -lrt
Memory location    : Please put data memory location here
                    (e.g. code in flash, data on heap etc)
seedcrc           : 0xe9f5
[0]crclist        : 0xe714
[0]crcmatrix      : 0x1fd7
[0]crcstate       : 0x8e3a
[0]crcfinal       : 0x65c5
Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 29306.176277 / GCC13.3.0 -O2 -DPERFORMANCE_RUN=1 -lrt / Heap
agustin@HP-Mersa:~/Escritorio/Facultad/Computacion/CoreMark/coremark$ cat run2.log
2K validation run parameters for coremark.
CoreMark Size      : 666
Total ticks        : 13817
Total time (secs): 13.817000
Iterations/Sec     : 28949.844395
Iterations         : 400000
Compiler version   : GCC13.3.0
Compiler flags     : -O2 -DPERFORMANCE_RUN=1 -lrt
Memory location    : Please put data memory location here
                    (e.g. code in flash, data on heap etc)
seedcrc           : 0x18f2
[0]crclist        : 0xe3c1
[0]crcmatrix      : 0x0747
[0]crcstate       : 0x8d84
[0]crcfinal       : 0x8091
Correct operation validated. See README.md for run and reporting rules.
agustin@HP-Mersa:~/Escritorio/Facultad/Computacion/CoreMark/coremark$

```

Rendimiento de Procesadores

El rendimiento del procesador depende de los siguientes parámetros:

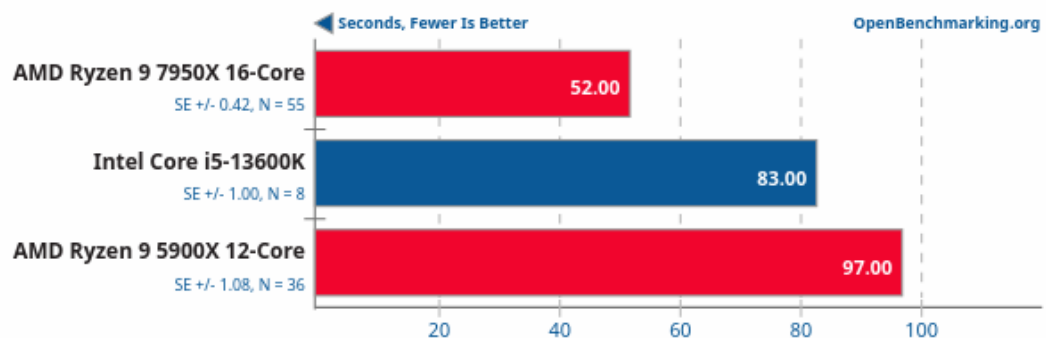
1. Frecuencia de la CPU (f_{CPU})
2. Periodo de la CPU (T_{CPU})
3. Ciclos por instrucción (CPI)
4. Número de instrucciones del programa
5. Multitarea

Actividad

¿Cuál es el rendimiento de estos procesadores para compilar el kernel de linux?

Timed Linux Kernel Compilation 6.8

Build: defconfig



Para determinar el rendimiento vamos a calcular el coeficiente de rendimiento según el tiempo de ejecución del programa en cada procesador

$$\eta_{\text{prog}} = \frac{1}{T_{\text{prog}}}$$

¿Cuál es la aceleración cuando usamos un AMD Ryzen 9 7950 X 16-Core?

Gracias a los datos de la página proporcionada se encuentra que la aceleración del procesador es de 400 MHz con una aceleración máxima de 5.8 GHz

cpu MHz : 400.000

CPU max MHz:	5881.0000
CPU min MHz:	400.0000

¿Cuál de ellos hace un uso más eficiente de la cantidad de núcleos que tiene? ¿Cuál es más eficiente en términos de costo (dinero y energía)?

	AMD RYZEN 9 5900X 12-CORE	INTEL CORE I5-13600K	AMD RYZEN 9 7950X 16-CORE
Core Count	12	14	16
Thread Count	24	20	32
CPU Clock *	3.7 GHz	5.1 GHz	4.5 GHz
Core Family	Zen 3	Raptor Lake	Zen 4
First Appeared	2020	2022	2022
Overall Percentile	57th	57th	72nd
Total OpenBenchmarking.org Results	59,300	10,397	58,170
Qualified Test Combinations	536	317	687
Performance Per Dollar	237	320	569
Affiliate Shopping Links *		Amazon	Amazon
More Details On OpenBenchmarking.org	Q	Q	Q
Remove From Comparison			

Utilizamos el Timed Linux kernel Compilation 6.8 ya que el procesador de intel i5 no contaba con la prueba de compilación 7.0

	Intel Core i5-13600K	AMD Ryzen 9 5900X 12-Core	AMD Ryzen 9 7950X 16-Core
costo	\$319 USD	\$295 USD	\$390 USD
cpus	14	12	16
tiempo	83	97	52
tiempo(n)	0.012	0.010	0.019
speedup	1	0.855	1.596
eficiencia(proc)	0.071	0.07125	0.09975
eficiencia(costo)	0.00713	0.00289	0.00409

¿Cuál de ellos hace un uso más eficiente de la cantidad de núcleos que tiene?

El AMD Ryzen 9 7950 X 16-Core. Al observar la eficiencia(proc), este procesador obtiene el valor más alto (0.09975), superando al Ryzen 9 5900X (0.07125) y al Intel Core i5-13600K (0.071).

¿Cuál es más eficiente en términos de costo (dinero y energía)?

El Intel Core i5-13600K. Según la eficiencia(costo), el procesador de Intel presenta la mejor relación con un valor de 0.00713, quedando por encima del Ryzen 9 7950 X (0.00409) y del Ryzen 9 5900X (0.00289).

Rendimiento ESP32

El análisis de rendimiento de la ESP32 se realizó mediante un código suministrado más adelante, a continuación se encuentran tablas comparativas de las diferentes frecuencias y sus efectos con el uso de enteros, floats y doubles.

Tabla rendimiento ESP32 Enteros

	Referencia	Mejora 1	Mejora 2
Frecuencia (GHz)	80	160	240
Tiempo(s)	10.01	4.98	3.31
Rendimiento	0.0999	0.200	0.302
Speedup	1	2.010	3.024
Eficiencia(frec)	0.0125	0.01254	0.0126
Aumento Freq	1	2	3
Tiempo estimado	10.01	5.005	3.336

El aumento de la frecuencia provoca de manera lineal una disminución en el tiempo de ejecución del programa, como se puede observar, duplicar la frecuencia tiene como consecuencia dividir a la mitad el tiempo de ejecución

Tabla rendimiento ESP32 Flotantes

	Referencia	Mejora 1	Mejora 2
Frecuencia (MHz)	80	160	240
Tiempo(s)	10.92	5.43	3.61
Rendimiento	0.0915	0.184	0.277
Speedup	1	2.011	3.024
Eficiencia(frec) s/f	0.0125	0.1256	0.126
Aumento Freq	1	2	3
Tiempo estimado	10.92	5.46	3.64

El ESP32 incluye una Unidad de Punto Flotante (FPU) por hardware de precisión simple (32 bits). Esto significa que las instrucciones matemáticas con float no requieren ser emuladas por software; el hardware dedicado las resuelve en muy pocos ciclos de reloj, logrando velocidades comparables a las de los enteros. Se observa que al duplicar la frecuencia, se divide a la mitad el tiempo, es un escalamiento lineal.

Tabla rendimiento ESP32 Double

	Referencia	Mejora 1	Mejora 2
Frecuencia (MHz)	80	160	240
Tiempo(s)	64.3	31.97	21.27
Rendimiento	0.0155	0.0312	0.047
Speedup	1	2.01	3.023
Eficiencia(frec)	0.0125	0.01256	0.0126
Aumento Freq	1	2	3
Tiempo estimado	64.3	32.15	21.433

El ESP32 no cuenta con un FPU por hardware para flotantes de doble precisión (64 bits). Las operaciones con double se calculan mediante librerías de software. El procesador debe ejecutar decenas o cientos de instrucciones estándar (enteras y lógicas) para resolver una sola suma de double, esto significa que el tiempo de ejecución será mucho mayor al que llevarán el manejo de enteros..

Al duplicar la frecuencia, el procesador ejecutará esas rutinas de emulación por software al doble de velocidad, por lo que el tiempo también se reducirá a la mitad.

A continuación está la devolución dada por el código en las diferentes pruebas:

Frecuencia de Procesador de 80 MHz

```
=====
Pruebas finalizadas.
=====
    BENCHMARK DE RENDIMIENTO - ESP32
=====
Frecuencia CPU: 80 MHz
Iteraciones por prueba: 71990000

1. Iniciando prueba de ENTEROS (uint32_t)...
   -> Completado en: 10011 ms (10.01 segundos)
   -> Resultado final: 71990000

2. Iniciando prueba de FLOATS (32 bits)...
   -> Completado en: 10920 ms (10.92 segundos)
   -> Resultado final: 33554432.00

3. Iniciando prueba de DOUBLES (64 bits)...
   -> Completado en: 64303 ms (64.30 segundos)
   -> Resultado final: 71997199.0910

=====
```

Frecuencia de procesador de 160 MHz

```
=====
    BENCHMARK DE RENDIMIENTO - ESP32
=====
Frecuencia CPU: 160 MHz
Iteraciones por prueba: 71990000

1. Iniciando prueba de ENTEROS (uint32_t)...
   -> Completado en: 4977 ms (4.98 segundos)
   -> Resultado final: 71990000

2. Iniciando prueba de FLOATS (32 bits)...
   -> Completado en: 5430 ms (5.43 segundos)
   -> Resultado final: 33554432.00

3. Iniciando prueba de DOUBLES (64 bits)...
   -> Completado en: 31966 ms (31.97 segundos)
   -> Resultado final: 71997199.0910

=====
```

Frecuencia de procesador de 240 MHz

```
=====
Pruebas finalizadas.
=====
    BENCHMARK DE RENDIMIENTO - ESP32
=====
Frecuencia CPU: 240 MHz
Iteraciones por prueba: 71990000

1. Iniciando prueba de ENTEROS (uint32_t)...
   -> Completado en: 3312 ms (3.31 segundos)
   -> Resultado final: 71990000

2. Iniciando prueba de FLOATS (32 bits)...
   -> Completado en: 3613 ms (3.61 segundos)
   -> Resultado final: 33554432.00

3. Iniciando prueba de DOUBLES (64 bits)...
   -> Completado en: 21272 ms (21.27 segundos)
   -> Resultado final: 71997199.0910

=====
```

Por último, se encuentra el código utilizado

```

#include <Arduino.h>

// Cantidad de iteraciones ajustada
const unsigned long ITERACIONES = 71990000UL;

void setup() {
    setCpuFrequencyMhz(240);
    Serial.begin(115200);
    while (!Serial) { delay(10); } // Esperar conexión serie

    delay(2000);

    Serial.println("=====");
    Serial.println("  BENCHMARK DE RENDIMIENTO - ESP32  ");
    Serial.println("=====");
    Serial.print("Frecuencia CPU: ");
    Serial.print(getCpuFrequencyMhz());
    Serial.println(" MHz");
    Serial.printf("Iteraciones por prueba: %lu\n\n", ITERACIONES);

    // -----
    // PRUEBA 1: Suma de Enteros (uint32_t)
    // -----
    Serial.println("1. Iniciando prueba de ENTEROS (uint32_t)...");

    volatile uint32_t suma_int = 0;
    unsigned long inicio_int = millis();

    for (unsigned long i = 0; i < ITERACIONES; i++) {
        suma_int += 1;
    }

    unsigned long tiempo_int = millis() - inicio_int;
    float segundos_int = tiempo_int / 1000.0;

    Serial.printf("  -> Completado en: %lu ms (%.2f segundos)\n", tiempo_int, segundos_int);
    Serial.printf("  -> Resultado final: %u\n\n", suma_int);

    // -----
    // PRUEBA 2: Suma de Flotantes (float - 32 bits)
    // -----
    Serial.println("2. Iniciando prueba de FLOATS (32 bits)...");

    volatile float suma_float = 0.0f;
    unsigned long inicio_float = millis();

    for (unsigned long i = 0; i < ITERACIONES; i++) {
        suma_float += 1.0001f;
    }

    unsigned long tiempo_float = millis() - inicio_float;
    float segundos_float = tiempo_float / 1000.0;

    Serial.printf("  -> Completado en: %lu ms (%.2f segundos)\n", tiempo_float, segundos_float);
    Serial.printf("  -> Resultado final: %.2f\n\n", suma_float);

    // -----
    // PRUEBA 3: Suma de Doble Precisión (double - 64 bits)
    // -----
    Serial.println("3. Iniciando prueba de DOUBLES (64 bits)...");

    volatile double suma_double = 0.0;
    unsigned long inicio_double = millis();

    for (unsigned long i = 0; i < ITERACIONES; i++) {
        suma_double += 1.0001;
    }

    unsigned long tiempo_double = millis() - inicio_double;
    float segundos_double = tiempo_double / 1000.0;

    Serial.printf("  -> Completado en: %lu ms (%.2f segundos)\n", tiempo_double, segundos_double);
    Serial.printf("  -> Resultado final: %.4f\n\n", suma_double);

    Serial.println("=====");
    Serial.println("Pruebas finalizadas.");
}

void loop() {
    // Queda vacío para ejecutar una sola vez en setup
}

```

Time Profiling

Ejecución paso a paso de la herramienta de análisis GPROF

```
Facultad/Computacion$ nano test_gprof.c
Facultad/Computacion$ nano test_gprof_new.c
Facultad/Computacion$
```

```
GNU nano 7.2
//test_gprof.c
#include<stdio.h>

void new_func1(void);

void func1(void)
{
    printf("\n Inside func1 \n");
    int i = 0;

    for(;i<0xffffffff;i++);
    new_func1();

    return;
}

static void func2(void)
{
    printf("\n Inside func2 \n");
    int i = 0;

    for(;i<0xffffffffaa;i++);
    return;
}

int main(void)
{
    printf("\n Inside main()\n");
    int i = 0;

    for(;i<0xfffff;i++);
    func1();
    func2();

    return 0;
}
```

```
GNU nano 7.2
//test_gprof_new.c
#include<stdio.h>

void new_func1(void)
{
    printf("\n Inside new_func1()\n");
    int i = 0;

    for(;i<0xfffffff;i++);

    return;
}
```

```
Facultad/Computacion$ gcc -Wall -pg test_gprof.c test_gprof_new.c -o test_gprof
```

```
agustin@HP-Mersa:~/Escritorio/Facultad/Computacion$ ls
clonada  ejerciciopull  parte5  test_gprof.c
CoreMark libro  test_gprof  test_gprof_new.c
```

```
agustin@HP-Mersa:~/Escritorio/Facultad/Computacion$ ./test_gprof
```

Inside main()

Inside func1

Inside new_func1()

Inside func2

```
agustin@HP-Mersa:~/Escritorio/Facultad/Computacion$ ls
clonada  ejerciciopull  libro  test_gprof  test_gprof_new.c
CoreMark gmon.out      parte5 test_gprof.c
agustin@HP-Mersa:~/Escritorio/Facultad/Computacion$
```

```
agustin@HP-Mersa:~/Escritorio/Facultad/Computacion$ gprof test_gprof gmon.out > analysis.txt
agustin@HP-Mersa:~/Escritorio/Facultad/Computacion$ ls
analysis.txt CoreMark      gmon.out  parte5  test_gprof.c
clonada      ejerciciopull libro      test_gprof  test_gprof_new.c
```

GNU nano 7.2

Flat profile:

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self s/call	total s/call	name
33.82	7.70	7.70	1	7.70	15.22	func1
33.03	15.22	7.52	1	7.52	7.52	func2
33.03	22.74	7.52	1	7.52	7.52	new_func1
0.13	22.77	0.03				main

% time the percentage of the total running time of the program used by this function.

cumulative seconds a running sum of the number of seconds accounted for by this function and those listed above it.

self seconds the number of seconds accounted for by this function alone. This is the major sort for this listing.

calls the number of times this function was invoked, if this function is profiled, else blank.

self ms/call the average number of milliseconds spent in this function per call, if this function is profiled, else blank.

total ms/call the average number of milliseconds spent in this function and its descendents per call, if this function is profiled, else blank.

name the name of the function. This is the minor sort for this listing. The index shows the location of the function in the gprof listing. If the index is in parenthesis it shows where it would appear in the gprof listing if it were to be printed.

^L

Copyright (C) 2012-2024 Free Software Foundation, Inc.

Copying and distribution of this file, with or without modification, are permitted in any medium without royalty provided the copyright notice and this notice are preserved.

^L

Call graph (explanation follows)

granularity: each sample hit covers 4 byte(s) for 0.04% of 22.77 seconds

index	% time	self	children	called	name
					<spontaneous>
[1]	100.0	0.03	22.74		main [1]
		7.70	7.52	1/1	func1 [2]
		7.52	0.00	1/1	func2 [3]

```
/Computacion$ gprof -a test_gprof gmon.out > analysis.txt
```

```
GNU nano 7.2
Flat profile:

Each sample counts as 0.01 seconds.
%   cumulative   self           self         total
time  seconds    seconds   calls   s/call   s/call   name
66.84    15.22    15.22         2      7.61    11.37  func1
33.03    22.74     7.52         1      7.52     7.52  new_func1
 0.13    22.77     0.03                main

%           the percentage of the total running time of the
time        program used by this function.

cumulative  a running sum of the number of seconds accounted
seconds     for by this function and those listed above it.

self        the number of seconds accounted for by this
seconds     function alone.  This is the major sort for this
            listing.

calls       the number of times this function was invoked, if
            this function is profiled, else blank.

self        the average number of milliseconds spent in this
ms/call     function per call, if this function is profiled,
            else blank.

total       the average number of milliseconds spent in this
ms/call     function and its descendents per call, if this
            function is profiled, else blank.

name        the name of the function.  This is the minor sort
            for this listing.  The index shows the location of
            the function in the gprof listing.  If the index is
            in parenthesis it shows where it would appear in
            the gprof listing if it were to be printed.

^L
Copyright (C) 2012-2024 Free Software Foundation, Inc.

Copying and distribution of this file, with or without modification,
are permitted in any medium without royalty provided the copyright
notice and this notice are preserved.

^L
Call graph (explanation follows)

granularity: each sample hit covers 4 byte(s) for 0.04% of 22.77 seconds

index % time    self  children   called    name
[1]   100.0      0.03  22.74      2/2      <spontaneous>
      15.22     7.52      2/2      main [1]
      -----
      15.22     7.52      2/2      func1 [2]
[2]   99.9      15.22   7.52       2      main [1]
      15.22     7.52       2      func1 [2]
```



```

GNU nano 7.2
Flat profile:

Each sample counts as 0.01 seconds.
 %   cumulative   self           calls   self   total    name
time  seconds    seconds             s/call  s/call  s/call  name
33.82    7.70      7.70              1    7.70   15.22  func1
33.03   15.22      7.52              1    7.52    7.52  func2
33.03   22.74      7.52              1    7.52    7.52  new_func1
 0.13   22.77      0.03                      0.03    0.03  main
^L
                                Call graph

granularity: each sample hit covers 4 byte(s) for 0.04% of 22.77 seconds

index % time    self  children    called    name
-----
[1]   100.0    0.03   22.74          1/1    <spontaneous>
      7.70    7.52          1/1    main [1]
      7.52    0.00          1/1    func1 [2]
      -----
      7.70    7.52          1/1    main [1]
[2]    66.8    7.70    7.52          1    func1 [2]
      7.52    0.00          1/1    new_func1 [4]
      -----
      7.52    0.00          1/1    main [1]
[3]    33.0    7.52    0.00          1    func2 [3]
      -----
      7.52    0.00          1/1    func1 [2]
[4]    33.0    7.52    0.00          1    new_func1 [4]
      -----
^L
Index by function name

      [2] func1
      [3] func2
      [1] main
      [4] new_func1

```

```

/Computacion$ gprof -p -b test_gprof gmon.out > analysis.txt
/Computacion$ nano analysis.txt

```

```

GNU nano 7.2                                analysis.txt
Flat profile:

Each sample counts as 0.01 seconds.
 %   cumulative   self           calls   self   total    name
time  seconds    seconds             s/call  s/call  s/call  name
33.82    7.70      7.70              1    7.70   15.22  func1
33.03   15.22      7.52              1    7.52    7.52  func2
33.03   22.74      7.52              1    7.52    7.52  new_func1
 0.13   22.77      0.03                      0.03    0.03  main

```

```

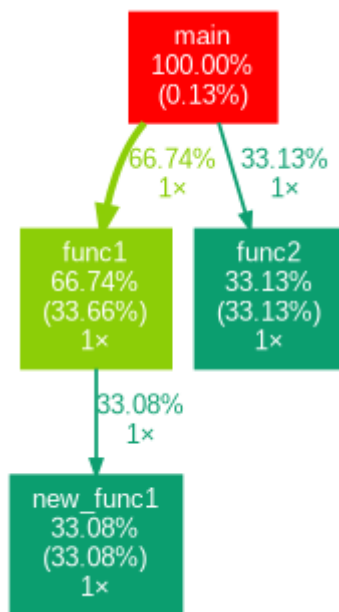
/Computacion$ gprof -pfunc1 -b test_gprof gmon.out > analysis.txt
/Computacion$ nano analysis.txt

```

```
GNU nano 7.2 analysis
Flat profile:

Each sample counts as 0.01 seconds.
 %   cumulative   self           calls   self   total   name
time   seconds    seconds             s/call   s/call   s/call   name
100.00      7.70      7.70              1      7.70    7.70    func1
```

Gráfico



Perf

```
agustin@HP-Mersa:~/Escritorio/Facultad/Computacion$ sudo perf record ./test_gprof

Inside main()

Inside func1

Inside new_func1()

Inside func2
[ perf record: Woken up 12 times to write data ]
[ perf record: Captured and wrote 3,507 MB perf.data (91344 samples) ]
agustin@HP-Mersa:~/Escritorio/Facultad/Computacion$
```

Samples: 91K of event 'cpu/cycles/P', Event count (approx.): 91387183084

Overhead	Command	Shared Object	Symbol
33,68%	test_gprof	test_gprof	[.] func1
33,09%	test_gprof	test_gprof	[.] new_func1
33,08%	test_gprof	test_gprof	[.] func2
0,13%	test_gprof	test_gprof	[.] main
0,00%	test_gprof	[kernel.kallsyms]	[k] srso_return_thunk
0,00%	test_gprof	[kernel.kallsyms]	[k] update_process_times
0,00%	test_gprof	[kernel.kallsyms]	[k] rtw_pci_write32
0,00%	test_gprof	[kernel.kallsyms]	[k] __rcu_read_unlock
0,00%	test_gprof	[kernel.kallsyms]	[k] timerqueue_add
0,00%	test_gprof	[kernel.kallsyms]	[k] dm_write_reg_func
0,00%	test_gprof	[kernel.kallsyms]	[k] crc32c_arch

Conclusiones

Si observamos el código, este tiene bucles for con el único objetivo de hacer tardar el programa, con límites de iteraciones enormes (como 0xffffffff, equivalente a más de 4 mil millones de iteraciones). Estos bucles no hacen nada útil, solo incrementan la variable *i* de forma repetitiva.

¿Cómo podría disminuir el tiempo de ejecución de este código ?

Tenemos 2 alternativas

- **Optimización del Código (Manual):** en este caso resulta obvio ver las razones del porqué se tarda tanto en ejecutar el programa, podemos acelerar la ejecución simplemente modificando los límites de iteración de los bucles for, pero este método resulta ineficiente en caso de programas de mayor complejidad y tamaño.
- **Optimización del Compilador (Automática):** Los compiladores modernos de C son capaces de detectar código ineficiente. Dado que la variable "i" se incrementa pero su valor final nunca se utiliza, el compilador puede catalogar todo el bucle como "código muerto" (dead code). Se puede ejecutar el código con la bandera -O3 para que el compilador elimine estos bucles:

```
gcc -Wall -O3 -pg test_gprof.c test_gprof_new.c -o test_gprof
```

IP, Ancho de Banda y Latencia

El ejercicio se realizó en una red lan utilizando una computadora de escritorio con windows y una Notebook con Linux mint.

IP's de los Equipos

NOTEBOOK: (192.168.1.3)

```

agustin@HP-Mersa:~$ ip -c a
1: lo: <LOOPBACK,UP,LOWER UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: wlo1: <BROADCAST,MULTICAST,UP,LOWER UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether b4:8c:9d:76:64:d7 brd ff:ff:ff:ff:ff:ff
    altname wlp1s0
    inet 192.168.1.3/24 brd 192.168.1.255 scope global dynamic noprefixroute wlo1
        valid_lft 75994sec preferred_lft 75994sec
    inet6 fe80::9780:7ee6:7615:9c6c/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
agustin@HP-Mersa:~$

```

PC DE ESCRITORIO:(192.168.1.99)

```

C:\Windows\system32\cmd.exe
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Agustin>ipconfig

Configuración IP de Windows

Adaptador de Ethernet Ethernet:

    Sufijo DNS específico para la conexión. . . :
    Vínculo: dirección IPv6 local. . . : fe80::7589:a409:19e1:cca9%3
    Dirección IPv4. . . . . : 192.168.1.99
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . . : 192.168.1.1

C:\Users\Agustin>

```

Medición de Ancho de Banda

Para ello se vincularon ambos equipos(la notebook estaba conectada a la red mediante wifi y la PC de escritorio a ethernet) mediante comandos y se midió el BW entre estas

```

C:\Users\Agustin\Desktop\iperf3.1.1_32>iperf3.exe -c 192.168.1.3
Connecting to host 192.168.1.3, port 5201
[ 4] local 192.168.1.99 port 50687 connected to 192.168.1.3 port 5201
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.00-1.00 sec  26.6 MBytes  222 Mbits/sec
[ 4] 1.00-2.00 sec  26.8 MBytes  225 Mbits/sec
[ 4] 2.00-3.00 sec  27.0 MBytes  227 Mbits/sec
[ 4] 3.00-4.00 sec  27.5 MBytes  231 Mbits/sec
[ 4] 4.00-5.00 sec  27.3 MBytes  229 Mbits/sec
[ 4] 5.00-6.00 sec  27.3 MBytes  229 Mbits/sec
[ 4] 6.00-7.00 sec  27.3 MBytes  228 Mbits/sec
[ 4] 7.00-8.00 sec  27.5 MBytes  231 Mbits/sec
[ 4] 8.00-9.00 sec  27.0 MBytes  227 Mbits/sec
[ 4] 9.00-10.00 sec 27.6 MBytes  231 Mbits/sec
-----
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.00-10.00 sec 272 MBytes  228 Mbits/sec
[ 4] 0.00-10.00 sec 272 MBytes  228 Mbits/sec
iperf Done.

C:\Users\Agustin\Desktop\iperf3.1.1_32>

```

Fast

NOTEBOOK:



Tu velocidad de internet es de

310 Mbps



[Mostrar más información](#)

PC DE ESCRITORIO:



Tu velocidad de internet es de

460 Mbps



Medición de Latencia

Sin Perturbaciones

Notebook:

```
64 bytes from 8.8.8.8: icmp_seq=107 ttl=118 time=14.6 ms
64 bytes from 8.8.8.8: icmp_seq=108 ttl=118 time=15.2 ms
64 bytes from 8.8.8.8: icmp_seq=109 ttl=118 time=14.4 ms
64 bytes from 8.8.8.8: icmp_seq=110 ttl=118 time=14.9 ms
^C
--- 8.8.8.8 ping statistics ---
110 packets transmitted, 110 received, 0% packet loss, time 109118ms
rtt min/avg/max/mdev = 13.563/14.995/41.242/2.568 ms
```

PC de Escritorio:

```
Respuesta desde 8.8.8.8: bytes=32 tiempo=13ms TTL=118
Respuesta desde 8.8.8.8: bytes=32 tiempo=13ms TTL=118
Respuesta desde 8.8.8.8: bytes=32 tiempo=13ms TTL=118
Respuesta desde 8.8.8.8: bytes=32 tiempo=13ms TTL=118
Respuesta desde 8.8.8.8: bytes=32 tiempo=13ms TTL=118
Respuesta desde 8.8.8.8: bytes=32 tiempo=13ms TTL=118

Estadísticas de ping para 8.8.8.8:
    Paquetes: enviados = 105, recibidos = 105, perdidos = 0
    (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 12ms, Máximo = 14ms, Media = 12ms
Control-C
^C
C:\Users\Agustin\Desktop\iperf3.1.1_32>
```

Con Perturbaciones:

Estas fueron provocadas en la PC de Escritorio mediante un comando

```
iperf3.exe -c [IP_DE_LINUX] -P 20 -t 60
```

Esto abrirá 20 conexiones paralelas durante 60 segundos, lo que provocará que los tiempos en ms se disparen.

```
64 bytes from 8.8.8.8: icmp_seq=17 ttl=118 time=59.7 ms
64 bytes from 8.8.8.8: icmp_seq=18 ttl=118 time=49.9 ms
64 bytes from 8.8.8.8: icmp_seq=19 ttl=118 time=70.5 ms
64 bytes from 8.8.8.8: icmp_seq=20 ttl=118 time=40.6 ms
64 bytes from 8.8.8.8: icmp_seq=21 ttl=118 time=45.6 ms
64 bytes from 8.8.8.8: icmp_seq=22 ttl=118 time=66.6 ms
64 bytes from 8.8.8.8: icmp_seq=23 ttl=118 time=54.9 ms
64 bytes from 8.8.8.8: icmp_seq=24 ttl=118 time=66.4 ms
64 bytes from 8.8.8.8: icmp_seq=25 ttl=118 time=44.7 ms
```